Explainable Machine Learning for Digital Marketing Campaign Performance Prediction

Predicting whether a campaign's next-day click-through rate will beat its benchmark, using an Explainable Boosting Machine

Programme	MSc in Artificial Intelligence
Module	Machine Learning (assignment report)
Student name	Jayawardena J.D.N.T.
Student ID	269122F
Submission date	29 October 2026
Source code	https://github.com/nimilaJay/ad-performance-ml
Live demo	https://ctr-dashboard.purplefield-4d57f34d.eastasia.azurecontainerapps.io
Demo video	drive.google.com/file/d/1m8TrZksWjLH3ZzdovIZgJySQriniz1L0/view

Abstract

Digital marketing teams review campaign performance every day and decide which campaigns need attention, but platform dashboards only describe what has already happened. This project builds a model that predicts whether a campaign's click-through rate (CTR) tomorrow will beat a benchmark for comparable campaigns, and explains each prediction. The main model is an Explainable Boosting Machine (EBM), a generalised additive model with pairwise interactions that is fitted by gradient boosting but remains fully interpretable. It is compared with logistic regression, CatBoost and a naive persistence rule on a chronologically held-out test period. The data is simulated: it was generated by a script, not collected from real advertising accounts, so the results show that the method works and are not evidence about real campaigns. The pipeline was developed and evaluated on a simulated dataset of 4,818 campaign-days from 90 campaigns; it accepts Google Ads and Meta Ads exports without code changes. On the test period the EBM reached a ROC-AUC of 0.893 (95% CI 0.832–0.936). This was indistinguishable from CatBoost (0.895) and significantly better than assuming tomorrow will look like today (difference +0.022, 95% CI 0.006 to 0.039). The EBM's learned shape functions described patterns in the simulated data, including CTR persistence, creative fatigue and a weekend dip, and also exposed an effect that the model averages away. A Streamlit dashboard, deployed on Microsoft Azure through a tested GitHub Actions pipeline, makes the predictions and explanations available to non-technical users.

Contents

1. Introduction . . . 4
1.1 Background and motivation . . . 4
1.2 Problem statement . . . 4
1.3 Objectives . . . 4

2. Dataset . . . 5
2.1 Dataset overview . . . 5
2.2 Unit of analysis and target variable . . . 5
2.3 Features . . . 6
2.4 Preprocessing . . . 6
2.5 Dataset size and split . . . 7
2.6 Data handling and import . . . 7

3. Algorithm selection: Explainable Boosting Machine . . . 7
3.1 How EBM works . . . 7
3.2 Why EBM was selected . . . 8
3.3 How EBM differs from standard models . . . 8
3.4 Eligibility . . . 9

4. Methodology . . . 9
4.1 Overview . . . 9
4.2 Exploratory analysis . . . 9
4.3 Chronological split . . . 10
4.4 Models and hyperparameter search . . . 10
4.5 Baseline . . . 11
4.6 Evaluation metrics . . . 11

5. Results . . . 11
5.1 Overall test performance . . . 11
5.2 Effect of hyperparameters . . . 13
5.3 Performance across segments . . . 13

6. Explainability and interpretation . . . 14
6.1 Global feature importance . . . 14
6.2 Learned shape functions . . . 14
6.3 Interactions . . . 16
6.4 Local explanations . . . 16
6.5 Partial dependence . . . 16
6.6 Permutation importance . . . 17
6.7 Does the model agree with domain knowledge? . . . 19

7. Front-end dashboard and deployment . . . 19
7.1 Dashboard . . . 19
7.2 Deployment and MLOps . . . 21
7.3 Deployment challenges . . . 22

8. Critical discussion . . . 23
8.1 Limitations of the model . . . 23
8.2 Data quality . . . 23
8.3 Risks of bias and unfairness . . . 24
8.4 Real-world impact and ethical considerations . . . 24

9. Conclusion and future work . . . 24

References . . . 24

Appendix A. Reproducing the results . . . 25

1. Introduction

1.1 Background and motivation

Online advertising budgets are now spread across several platforms, and most of that spend is managed day by day. A marketing team running dozens of campaigns on Google Ads and Meta Ads has to decide each morning which campaigns to leave alone, which to refresh with new creative, and which to pause or rebalance. Click-through rate is one of the first signals it looks at. CTR reacts quickly, it is available for every campaign type, and a falling CTR is often the earliest visible sign that an advert has worn out or is reaching the wrong audience.

The difficulty is that daily CTR is noisy. It moves with the day of the week, with auction competition, with budget changes and with the age of the creative. Platform dashboards report these movements after the fact but do not say whether yesterday's dip is likely to continue. A forward-looking warning would help a team prioritise its attention, but only if it comes with a reason. A score saying that a campaign is at risk is of little use unless the team can see whether the cause is a long decline in engagement, an old creative or simply that tomorrow is a Saturday. Most published work on click prediction targets individual ad impressions at very large scale (Richardson, Dominowska and Ragno, 2007; McMahan et al., 2013). This project works one level up, at the level of the campaign-day, where the user is a marketer rather than an ad server.

1.2 Problem statement

The task is framed as binary classification. For each campaign at the end of day t, using only data available at that point, the model estimates the probability that the campaign's CTR on day t+1 will exceed a benchmark for campaigns of the same platform and objective. A positive prediction means the campaign is expected to beat the benchmark; a negative prediction flags it as at risk of underperforming. The output is meant as decision support. Its explanations describe statistical associations in historical data, not causal effects.

1.3 Objectives

1 Assemble a campaign-day dataset containing the features listed in the project brief, and define a fair target that leaks no future information.

2 Identify the factors associated with next-day CTR through exploratory analysis.

3 Train an Explainable Boosting Machine and compare it with logistic regression, CatBoost and a persistence baseline.

4 Evaluate the models with precision, recall, F1-score and ROC-AUC on a chronologically held-out test period, with confidence intervals.

5 Explain the EBM globally and for individual predictions, and check its behaviour against domain knowledge.

6 Build an interactive dashboard in which marketing users can enter campaign data and see predictions and explanations.

Section 2 describes the dataset and its preparation, Section 3 the choice of algorithm and Section 4 the method. Results are reported in Section 5 and interpreted in Section 6. Section 7 presents the dashboard, Section 8 discusses limitations and ethical issues, and Section 9 concludes.

2. Dataset

2.1 Dataset overview

The data used here is simulated. Access to a real advertising account had not been secured when this report was written, so the dataset was generated by `src/generate_data.py`. It does not come from real campaigns. The results in Sections 5 and 6 therefore show that the method works end to end and behaves sensibly; they are not evidence about the performance of real campaigns.

The generator sets typical CTR and CPM by platform and objective, then adds day-of-week effects, a learning period, creative fatigue, budget effects and random drift. Patterns of this kind reported later were therefore written into the data, so finding them is a check on the method, not a discovery about advertising.

The analysis uses a campaign-day dataset stored in `data/raw/campaign_daily.csv`. Each record contains a date, campaign identifier, platform, objective, spend, impressions and clicks. The pipeline validates these fields before building features and labels. Results in this report describe the simulated dataset and the held-out test period.

The simulated dataset contains 90 campaigns running between 11 January 2025 and 30 October 2025, 44 labelled Google Ads and 46 labelled Meta Ads, across five objectives: Awareness, Traffic, Engagement, Leads and Sales. Section 2.4 describes cleaning and Section 2.5 defines the chronological split used for evaluation.

2.2 Unit of analysis and target variable

Each row describes one campaign at the end of one day (t) and is built only from data up to and including that day. The target, *above_benchmark_next_day*, equals 1 when the campaign's CTR on day t+1 is higher than its benchmark and 0 otherwise. Two choices shape this definition.

First, the benchmark is set per platform and objective. CTR levels differ by almost an order of magnitude between, for example, Google lead-generation campaigns and Meta awareness campaigns (Table 1), so a single benchmark would mostly measure campaign type rather than performance. Each benchmark is the median daily CTR of all campaign-days in that group during the training period only. Fixing it before any validation or test data is seen keeps future information out of the labels, and using the median gives roughly balanced classes in training.

Second, a day is only labelled if day t+1 has at least 100 impressions. Below that, a handful of clicks can move CTR by several percentage points and the label would mostly reflect noise.

Table 1. Benchmark CTR per platform and objective (median daily CTR in the training period).

Platform	Awareness	Engagement	Leads	Sales	Traffic
Google Ads	0.54%	1.70%	4.41%	3.37%	3.19%
Meta Ads	0.45%	0.84%	0.78%	0.74%	1.17%

2.3 Features

Table 2 lists the 16 features. They cover the factors named in the project brief: platform, objective, campaign duration, historical spend, previous impressions and clicks, and previous CTR. CTR history is expressed as a *CTR index*, the campaign's CTR divided by its benchmark, so that a value of 1.0 means "at benchmark" for any campaign type. This puts Google and Meta campaigns on a common scale and makes the learned effects easier to read. The index is computed over the last day, the last three days and the last seven days, together with a trend term. Rolling windows are measured in calendar days, so a paused day counts as a day without delivery rather than being skipped. The day of week refers to the day being predicted, which is known in advance.

Table 2. Model features. Impressions and clicks each contribute two features, giving 16 in total.

Group	Feature	Definition
Context	Advertising platform	Google Ads or Meta Ads
	Campaign objective	Awareness, Traffic, Engagement, Leads or Sales
	Day of week	Day of the week being predicted (t+1)
	Campaign age	Days since the campaign's first day (campaign duration)
Spend	Spend, last day	Spend on day t
	Average daily spend, 7 days	Mean daily spend over days t−6 to t
	Cumulative spend	Total spend since the campaign started
	CPM, 7 days	Cost per 1,000 impressions over the last 7 days
Delivery	Impressions and clicks, last day	Impressions and clicks on day t
	Average daily impressions and clicks, 7 days	Means over days t−6 to t
CTR history	CTR index, 1, 3 and 7 days	CTR over the window divided by the benchmark
	CTR trend	One-day index minus seven-day index

2.4 Preprocessing

Cleaning (`src/data.py`) parses dates and numbers, removes total rows from exports, drops impossible records (negative values, or more clicks than impressions), merges duplicate campaign-days and fills gaps inside each campaign's date range with zero-delivery days. On the input dataset, the raw file had 4,933 rows; cleaning merged 25 duplicates, found 0 invalid values and filled 104 gap days, giving a daily panel of 5,012 campaign-days. Removing days without a next-day label (the last day of each campaign and days followed by fewer than 100 impressions) and days with no delivery in the previous week left 4,818 labelled rows.

Encoding and scaling depend on the model. The EBM handles platform, objective and day of week as nominal features and bins the numeric features, so it needs neither one-hot encoding nor scaling. CatBoost also handles categorical features natively, using ordered target statistics. For logistic regression the categorical features are one-hot encoded, the heavy-tailed volume features (spend, impressions, clicks, campaign age and CPM) are log-transformed, and all numeric features are standardised, with median imputation and a missing-value indicator. On a day with no delivery the one-day CTR is undefined, so the one-day index falls back to the seven-day index.

2.5 Dataset size and split

The final dataset has 4,818 labelled campaign-days, well above the target of 1,000 in the project brief. Table 3 shows how it divides into the chronological training, validation and test sets described in Section 4.3.

Table 3. Chronological split.

Split	Days predicted	Rows	Campaigns	Above benchmark
Training	13 Jan 2025 – 17 Aug 2025	3,383	67	50.2%
Validation	18 Aug 2025 – 24 Sep 2025	716	30	54.2%
Test	25 Sep 2025 – 30 Oct 2025	719	26	55.2%

2.6 Data handling and import

The project uses only aggregated, campaign-level statistics: daily spend, impressions and clicks. It does not use user-level data, demographics or any other personal information, so no individual can be identified from the dataset. Account exports should come only from accounts the author is authorised to access, with the account owner's permission. Campaign names can be commercially sensitive, so the conversion tool replaces them with one-way hashed identifiers, and account exports are excluded from version control.

To import additional data, daily campaign reports can be exported from Google Ads (a campaign report segmented by day, with cost, impressions and clicks) and from Meta Ads Manager (the campaign table broken down by day, with amount spent, impressions and link clicks), then converted with:

```
python -m src.data --export google.csv "Google Ads" --export meta.csv "Meta Ads"
```

The converter copes with the title lines that Google Ads places above the header, total rows, currency symbols and thousands separators. Retraining then needs no code changes.

3. Algorithm selection: Explainable Boosting Machine

3.1 How EBM works

An EBM is a generalised additive model with optional pairwise interactions, sometimes called a GA^{2}M (Lou, Caruana and Gehrke, 2012; Lou et al., 2013). For binary classification it models the log-odds of the positive class as

$$\log[\, p / (1 - p) \,] = \beta_0 + \Sigma_j f_j(x_j) + \Sigma_{i<j} f_{ij}(x_i, x_j)$$

where each f_j is an arbitrary learned function of one feature and each f_{ij} a function of a pair of features. Because the model is a sum, any prediction can be split exactly into one contribution per term.

What separates EBM from classical GAMs, which fit smooth splines (Hastie and Tibshirani, 1990), is how the functions are learned. The InterpretML implementation (Nori et al., 2019) uses cyclic gradient boosting. It visits the features one at a time in a round-robin, fits a very small tree to the current residuals using only that feature, and adds it to that feature's function with a low learning rate. Because each step sees a single feature, the effect of correlated features is shared out gradually instead of being claimed by whichever feature a tree happens to split on first. After thousands of rounds each function is stored as a lookup table over bins of the feature. Pairwise interactions are then chosen with the FAST

heuristic and fitted to the remaining residuals in the same way. Finally, the whole procedure is repeated on bootstrap samples of the training data (14 "outer bags" by default) and averaged, which lowers variance and gives an uncertainty band for every learned function.

3.2 Why EBM was selected

The task needs a model that can capture non-linear effects. The relationship between recent and future CTR levels off at high values, fatigue only sets in after the first week or so, and day-of-week effects are categorical rather than linear. At the same time the output is meant to be acted on by marketers, who need to know why a campaign is flagged. Post-hoc methods such as SHAP or LIME can describe a black-box model, but they are approximations whose output depends on the explainer and its settings, and Rudin (2019) argues that where an interpretable model performs as well, it should be preferred. EBMs have been shown to come close to the accuracy of gradient-boosted trees on tabular data (Caruana et al., 2015; Nori et al., 2019), and their explanations are the model itself rather than a description of it. The dataset also suits the method: a few thousand rows of mixed numeric and categorical features, on which an EBM trains in about a second.

3.3 How EBM differs from standard models

Table 4 compares EBM with the standard models named in the assignment guidelines and with CatBoost, which is used here as the accuracy reference.

Table 4. EBM compared with standard models.

Model	How it predicts	Non-linear effects	How it is explained	Weakness for this task
Decision tree	A sequence of if-then splits	Yes, as steps	Read the path to a leaf	Unstable; deep trees become unreadable, shallow ones are inaccurate
Logistic regression	Weighted sum of features	No (unless engineered)	One coefficient per feature	Cannot follow curved or saturating effects; unstable when features are correlated
k-nearest neighbours	Vote of the most similar past cases	Yes	Only by showing neighbours	No model of feature effects; sensitive to scaling
CatBoost (gradient-boosted trees)	Sum of hundreds of deep trees	Yes, with complex interactions	Needs post-hoc tools	Black box: explanations are approximations
EBM	Sum of learned per-feature and pairwise functions	Yes, any shape per feature	Exact: plot each function; add up contributions	Interactions limited to pairs

Compared with logistic regression, EBM keeps the additive structure but replaces each coefficient with a learned curve, so a feature can matter a great deal in one range and not at all in another. Compared with a decision tree, it does not split the data into ever smaller regions; each feature's effect is estimated from all of the data and stays readable however many features there are. k-NN stores no model at all, so it cannot say which features drove a prediction. Compared with a general gradient-boosting model such as CatBoost (Prokhorenkova et al., 2018), EBM restricts itself to single-feature and pairwise terms. That restriction is what makes it interpretable, and Section 5 tests whether it costs accuracy.

3.4 Eligibility

The assignment requires an algorithm that was not covered in lectures. EBM was not among the algorithms taught in the module [confirm with the module leader before submission]. Logistic regression appears in this project only as a comparison model.

4. Methodology

4.1 Overview

Figure 1 summarises the workflow, which follows the steps set out in the project brief. Everything is scripted in Python using pandas, scikit-learn (Pedregosa et al., 2011), InterpretML and CatBoost, and a full run from the raw file to the final figures takes under a minute on a laptop.

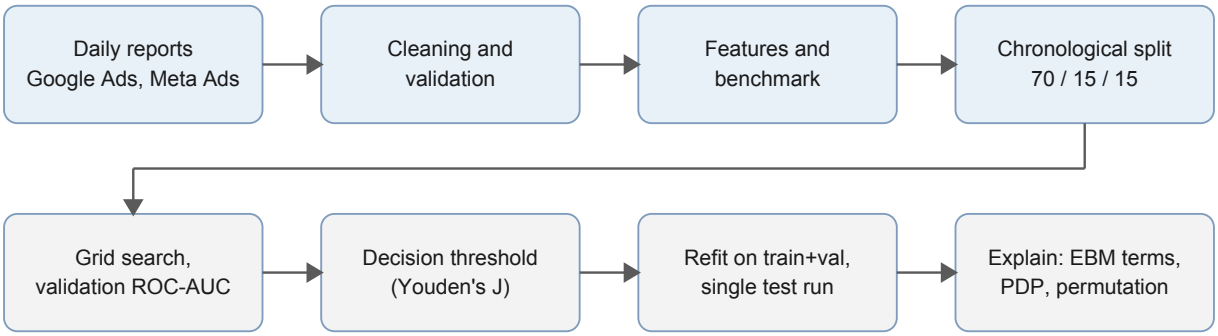


Figure 1. Modelling workflow. Blue steps prepare the data; grey steps train, evaluate and explain the models.

4.2 Exploratory analysis

Four findings from the exploratory analysis (`notebooks/01_eda.ipynb`) shaped the modelling. First, CTR levels differ strongly by objective and platform (Figure 2a), which motivated the per-group benchmark. Second, CTR is persistent: the Spearman correlation between a campaign's CTR index on one day and on the next is 0.67, and 0.66 for the seven-day index (Figure 2b), so a rule that simply assumes tomorrow will look like today is a demanding baseline. Third, CTR rises over the first week of a campaign and then declines with age on both platforms (Figure 2c), the expected signature of a learning phase followed by creative fatigue. Fourth, the weekly pattern runs in opposite directions on the two platforms (Figure 2d): Google campaigns are stronger on weekdays and Meta campaigns at weekends.

The share of campaign-days above benchmark also drifts upwards over time, from 50% in training to 54% in validation and 55% in test. This is one reason for testing on a later period rather than a random sample.



Figure 2. Exploratory analysis: (a, top left) daily CTR by objective and platform; (b, top right) CTR index today against tomorrow; (c, bottom left) median next-day CTR index by campaign age; (d, bottom right) median CTR index by day of week.

4.3 Chronological split

Rows are ordered by the date being predicted. The oldest 70% form the training set, the next 15% the validation set and the newest 15% the test set (Table 3). Nothing is shuffled, so every evaluation is on days later than anything the model was fitted on, which is how the model would be used in practice. A random split would let the model see the same campaign on the days either side of a test day and would overstate performance. As a check, the features for 60 randomly chosen rows were rebuilt from data cut off at day t; they were identical to the stored features, confirming that no future information reaches the inputs.

4.4 Models and hyperparameter search

Each model was tuned by grid search: every configuration was trained on the training set and scored by ROC-AUC on the validation set (Table 5). For EBM the search covered the number of pairwise interactions (0, 5 or 10, where 0 gives a purely additive model), the maximum number of bins per feature and the minimum number of samples per leaf, with the learning rate kept at the library default. For logistic regression it covered the inverse regularisation strength C and class weighting, and for CatBoost the tree depth, learning rate and L2 regularisation, with the number of trees set by early stopping on the validation set.

Table 5. Hyperparameter grids (all combinations were tried) and the selected values.

Model	Values searched	Selected
EBM	interactions: 0, 5, 10; max_bins: 32, 128; min_samples_leaf: 4, 20; learning_rate = 0.015; 14 outer bags	interactions = 10, max_bins = 128, min_samples_leaf = 4
Logistic regression	C: 0.01, 0.1, 1, 10; class_weight: none, balanced	C = 10, class_weight = balanced
CatBoost	depth: 4, 6; learning_rate: 0.03, 0.1; l2_leaf_reg: 3, 10; up to 2,000 trees with early stopping (100 rounds)	depth = 6, learning_rate = 0.03, l2_leaf_reg = 10

The decision threshold for each model was chosen on the validation set by maximising Youden's J statistic, sensitivity plus specificity minus one (Youden, 1950). An earlier version maximised F1-score instead, which pushed every threshold down to about 0.25 and meant the models almost never flagged an underperforming campaign, the main purpose of the tool. After tuning, each model was refitted on the training and validation sets together and evaluated once on the test set.

4.5 Baseline

A persistence baseline predicts that a campaign will beat its benchmark tomorrow if it beat it today (one-day CTR index above 1), and uses the one-day index as its ranking score. Given the persistence seen in the exploratory analysis, a model is only worth having if it beats this rule.

4.6 Evaluation metrics

Precision, recall and F1-score are reported with "above benchmark" as the positive class, as specified in the project brief. Because the practical aim is to catch underperforming campaigns, specificity is also reported, labelled *at-risk recall*: the share of below-benchmark days that the model flags. ROC-AUC and PR-AUC summarise ranking quality across all thresholds, and the Brier score measures the accuracy of the predicted probabilities. Days from the same campaign are correlated, so 95% confidence intervals were obtained by resampling whole campaigns rather than individual days (a cluster bootstrap; Efron and Tibshirani, 1993). The test set contains only 26 campaigns, so the intervals are fairly wide. Differences between models were assessed on paired resamples of the same campaigns.

5. Results

5.1 Overall test performance

Table 6 summarises the test-set results. The three trained models ranked campaign-days almost equally well: ROC-AUC was 0.893 for EBM, 0.895 for CatBoost and 0.892 for logistic regression, against 0.871 for the persistence baseline. The confidence intervals overlap heavily, which is expected with 26 test campaigns, so the paired comparisons in Table 7 are more informative.

Table 6. Test-set results (719 campaign-days, 26 campaigns). Thresholds were fixed on the validation set; the persistence rule uses a CTR index of 1.

Model	Thr.	Precision	Recall	F1	At-risk recall	ROC-AUC (95% CI)	PR-AUC	Brier
EBM	0.51	0.827	0.831	0.829	0.786	0.893 (0.832–0.936)	0.906	0.132
CatBoost	0.58	0.856	0.778	0.815	0.839	0.895 (0.833–0.937)	0.908	0.131
Logistic Regression	0.57	0.858	0.728	0.787	0.851	0.892 (0.834–0.933)	0.905	0.133
Persistence	1.00	0.805	0.824	0.814	0.755	0.871 (0.811–0.920)	0.889	–

Table 7. Paired differences in test ROC-AUC (cluster bootstrap, 1,000 resamples).

Comparison	Difference in ROC-AUC	95% CI	Resamples favouring EBM
EBM − persistence	+0.022	+0.006 to +0.039	99.9%
EBM − CatBoost	-0.002	-0.006 to +0.003	23.4%
EBM − Logistic Regression	+0.001	-0.003 to +0.006	75.0%

The EBM's advantage over persistence (+0.022) has an interval that excludes zero, so the model adds real information beyond yesterday's CTR. Its differences from CatBoost (-0.002) and logistic regression (+0.001) do not; in other words, the glass-box model gave up no measurable accuracy relative to the black-box reference.

At the chosen thresholds the models made different trade-offs. EBM had the highest recall of the three trained models (0.831) and the highest F1-score (0.829), and caught 79% of below-benchmark days. Logistic regression and CatBoost settled on higher thresholds and flagged more of the at-risk days (85% and 84%) at the cost of recall on the positive class. The persistence rule's F1-score (0.814) is close to the models', which deserves to be said plainly: for a simple yes/no decision the models add only a little over assuming that tomorrow will resemble today. Their larger contribution lies in ranking, in well-calibrated probabilities, which persistence cannot provide, and in the explanations.

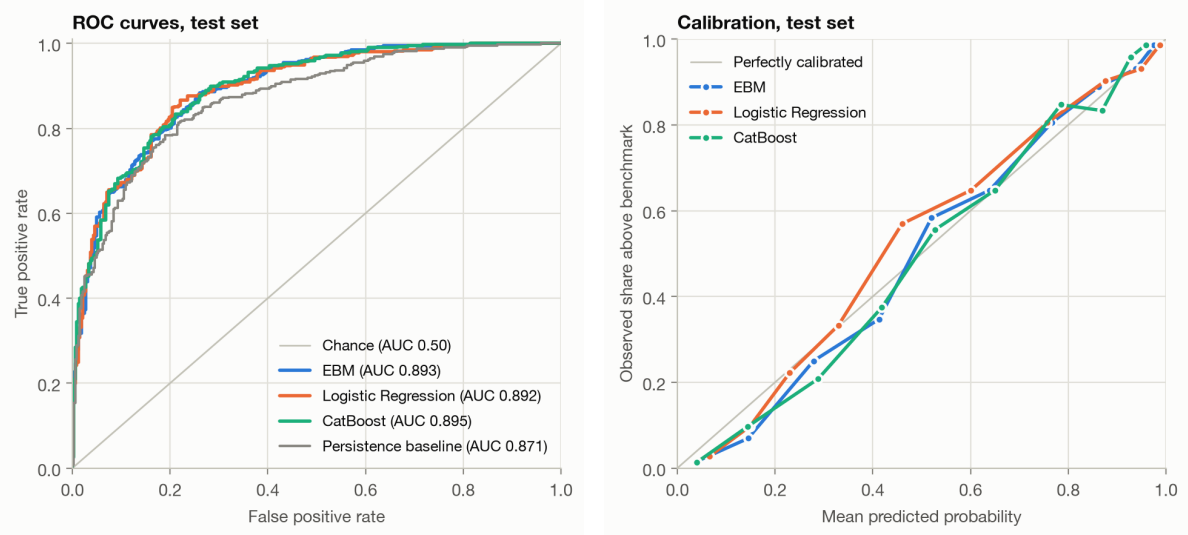


Figure 3. Test-set ROC curves (left) and calibration curves (right).

Figure 3 shows that the curves of the three trained models lie almost on top of one another and above the persistence curve across most of the range. The calibration curves follow the diagonal closely (Brier scores 0.131–0.133), so a predicted 70% chance of beating the benchmark can be read roughly at face value. Figure 4 shows the confusion matrices at the chosen thresholds.

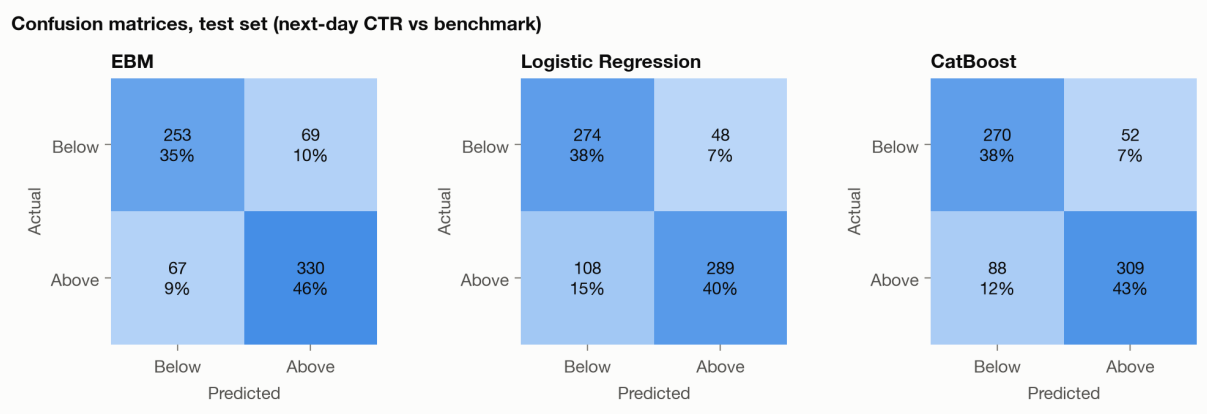


Figure 4. Test-set confusion matrices at the validation-chosen thresholds.

Validation ROC-AUC was lower than test ROC-AUC for every model (EBM 0.834 against 0.893). Part of this comes from refitting on more recent data, but the persistence rule, which involves no training at all, improved by a similar amount (0.812 to 0.871). Most of the gap therefore reflects a test period that happened to be easier to predict. The test figures should be read together with their intervals, and the validation figures give a fair idea of performance in a less favourable period.

5.2 Effect of hyperparameters

Hyperparameters mattered little. The best EBM used 10 interactions, but the purely additive model scored only 0.0010 lower in validation ROC-AUC (0.8328 against 0.8338). Most of the signal is carried by main effects, which is convenient, because main effects are the easiest terms to explain. Logistic regression preferred weak regularisation ($C = 10$) and CatBoost a depth of 6 with a learning rate of 0.03. All models trained quickly, taking on average 1.1 s per EBM fit and 0.5 s per CatBoost fit.

5.3 Performance across segments

Performance was broadly similar on the two platforms, with an EBM ROC-AUC of 0.910 on Google Ads and 0.878 on Meta Ads (Table 8). It varied more by objective, from 0.814 to 0.971, although the smallest group has only 57 test rows and its figure is noisy. The weak spot is at-risk recall for Sales campaigns (0.605): 75% of Sales days in the test period were above benchmark, and a single threshold shared by all groups flags fewer of the minority of bad days. A threshold per segment, or a benchmark that is updated over time, would address this.

Table 8. EBM test performance by platform and by objective.

Segment	Test rows	Above benchmark	ROC-AUC	F1	At-risk recall
Google Ads	302	57%	0.910	0.848	0.808
Meta Ads	417	54%	0.878	0.815	0.771
Awareness	112	48%	0.849	0.746	0.741
Engagement	119	40%	0.843	0.755	0.817
Leads	261	56%	0.918	0.855	0.819
Sales	170	75%	0.814	0.866	0.605
Traffic	57	40%	0.971	0.818	0.912

6. Explainability and interpretation

6.1 Global feature importance

EBM term importance is the mean absolute contribution of a term to the log-odds over the training data (Figure 5). The three CTR-index features dominate (seven-day 0.473, one-day 0.384, three-day 0.379), followed at a distance by campaign age (0.105), day of week (0.100) and the CTR trend (0.092). Clicks and spend carry small effects. Platform has almost no main effect (+0.004 and -0.003 log-odds for Google and Meta), which is exactly what should happen: the per-group benchmark already absorbs platform differences, so the model only has to learn how a campaign moves relative to its peers. Of the 26 terms in the final model, 10 are pairwise interactions, none with an importance above 0.042.

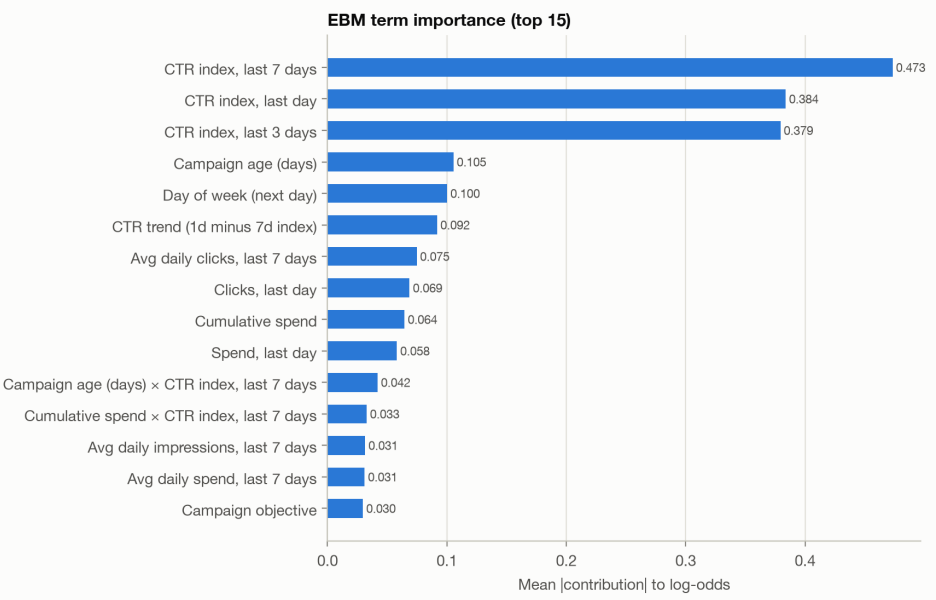


Figure 5. EBM term importance (15 most important terms).

6.2 Learned shape functions

Figure 6 shows what the EBM learned for its most important features. Each panel plots the contribution to the log-odds against the feature value; the shaded band shows the variation across the bagged models.

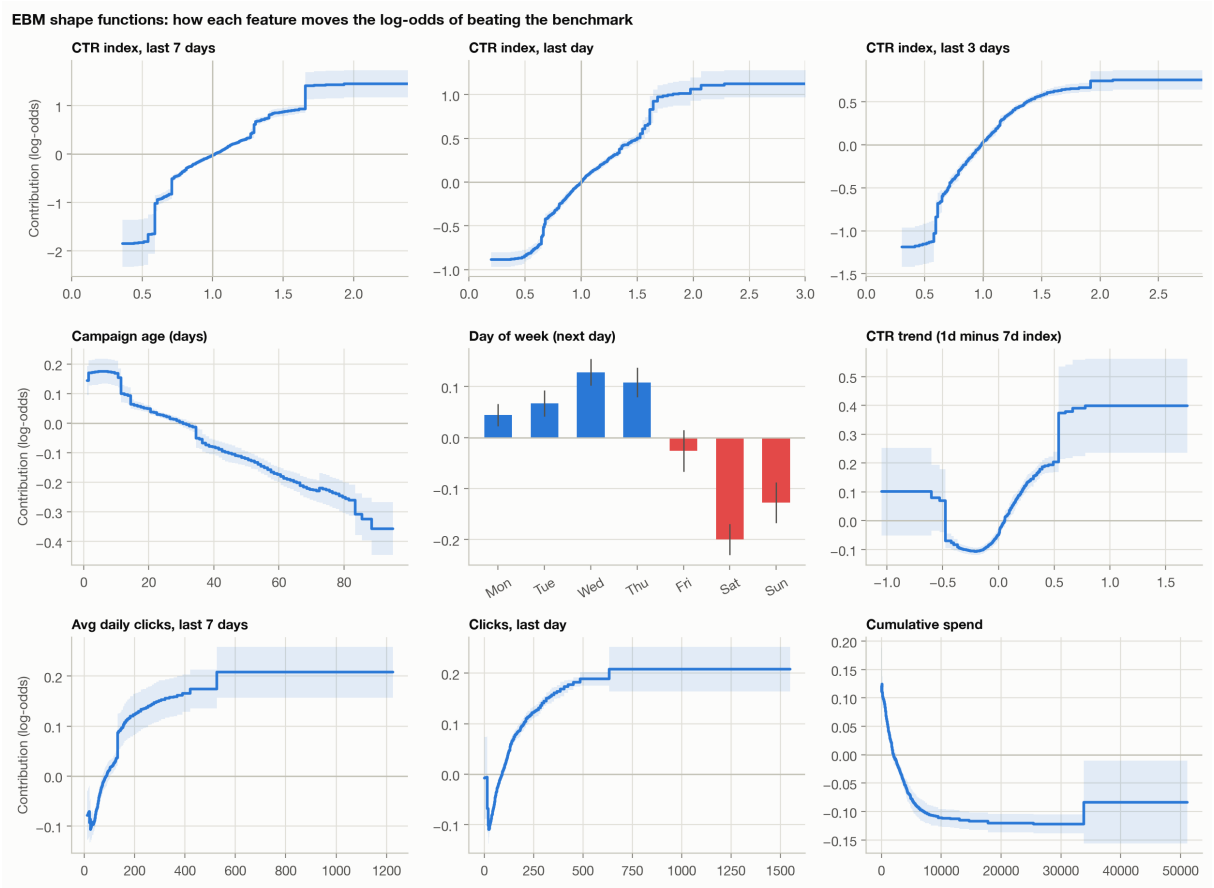


Figure 6. EBM shape functions for the nine most important main effects. Vertical lines mark a CTR index of 1 (at benchmark).

- **CTR index.** For all three windows the contribution rises steadily and crosses zero at an index of about 1.02 for the seven-day window, close to the point at which a campaign is exactly at its benchmark. That point was not given to the model; it was learned from the data. The curves flatten above an index of roughly 1.6 to 2, so a campaign far above benchmark gains little from being further above it.
- **Campaign age.** Young campaigns receive a small positive contribution (+0.14), after which the effect declines almost linearly to about -0.36 by day 88. This is the shape of creative fatigue.
- **Day of week.** Wednesday (+0.13) and Thursday (+0.11) are the strongest days and Saturday (-0.20) and Sunday (-0.13) the weakest.
- **CTR trend.** A sharp recent rise above the seven-day level raises the probability and a modest dip lowers it. Very large drops receive a slightly positive contribution, which hints at a rebound after one-off bad days, but this part of the curve rests on few observations and has a wide band.
- **Cumulative spend** has a falling curve, largely because it is correlated with campaign age. It should not be read as "spending more makes CTR worse".

6.3 Interactions

The strongest interaction pairs campaign age with the seven-day CTR index (Figure 7). A weak recent CTR is penalised more heavily in an older campaign than in a young one, while young campaigns with a low index receive a small upward correction. This fits the idea that a new campaign's poor start partly reflects the platform's learning phase and tends to recover, whereas poor CTR in an old campaign reflects fatigue and persists. The effect is real but small, in line with the small gain from interactions in Section 5.2.

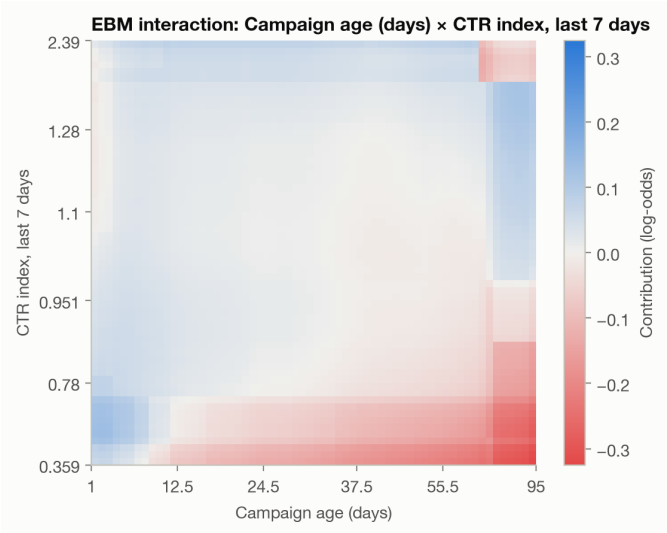


Figure 7. Strongest EBM interaction: campaign age × seven-day CTR index. Blue raises and red lowers the log-odds. Axes are in bins of the features.

6.4 Local explanations

Because the EBM is additive, each prediction breaks down exactly into an intercept plus one contribution per term, and the code checks that these add up to the predicted log-odds for every test row. Figure 8 shows two test cases. In the first, campaign cmp_040 on 18 October 2025 (predicted probability 0.3%), all three CTR-index terms are strongly negative because the campaign had been running at about 0.53 times its benchmark over the previous week; campaign age and the coming Saturday add further negative weight. The campaign did fall below its benchmark.

The second case is the most confident error in the test set. Campaign cmp_065 was given a 98% chance of beating its benchmark on 16 October 2025, because its CTR had been running at 1.64 times the benchmark over the previous week and 2.72 times on the last day. The next day its CTR fell to 0.775%, just below the benchmark of 0.776% and 39% lower than its own seven-day average. The explanation makes the error easy to understand: nothing in the campaign's history pointed to the drop, so it was most likely a one-off shock that no model based on past performance could foresee.

6.5 Partial dependence

Partial dependence plots (Friedman, 2001) show the average predicted probability as one feature is varied while the others keep their observed values, and they can be computed for any model. Figure 9 compares the three models. EBM and CatBoost produce almost the same curves, including the flattening at high CTR index, which suggests both have found the same underlying relationship. Logistic regression is limited to smooth monotone curves. For campaign age this exaggerates the effect: its partial dependence falls from 0.76 at the start of a campaign to 0.42 by day 87, whereas the EBM's falls only from 0.59 to 0.51 and CatBoost's from 0.60 to 0.53.

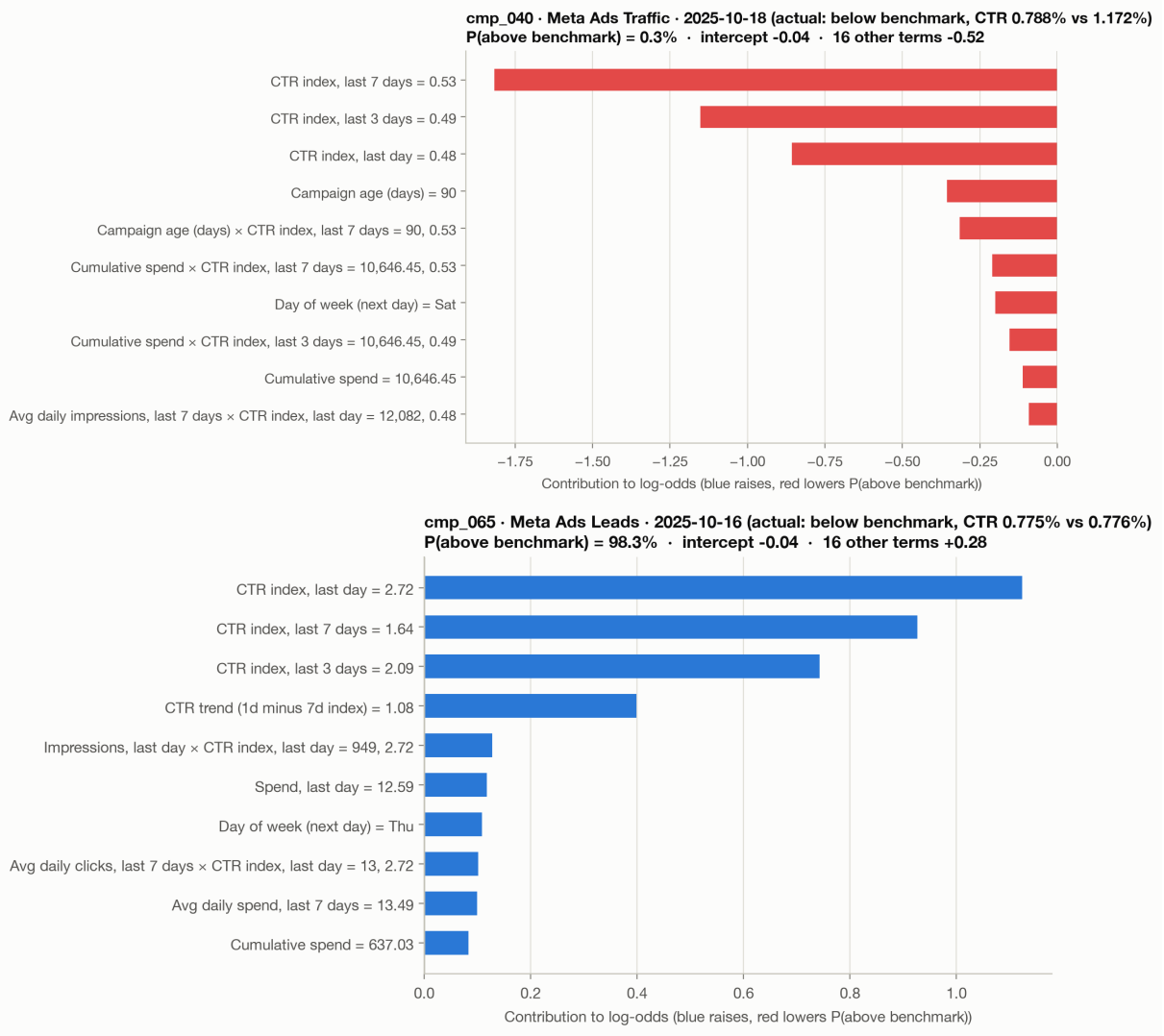


Figure 8. Local explanations for two test cases: a correctly flagged at-risk campaign (top) and the most confident misclassification (bottom). Blue bars raise and red bars lower the probability.

6.6 Permutation importance

Permutation importance measures how much test ROC-AUC drops when one feature is shuffled (Breiman, 2001). For EBM and CatBoost it confirms the global picture: the CTR-index features account for almost all of the drop and everything else is close to zero (Figure 10). Logistic regression behaves differently. It relies heavily on last-day spend (0.075) and campaign age (0.030), features the other two models largely ignore. The volume features are strongly correlated with each other, and a linear model can only approximate the curved CTR effects by leaning on correlated proxies. A marketer reading the logistic regression could easily conclude that spend drives CTR, which the other models do not support. This is a practical argument for EBM: its explanations are not only exact but also more stable.

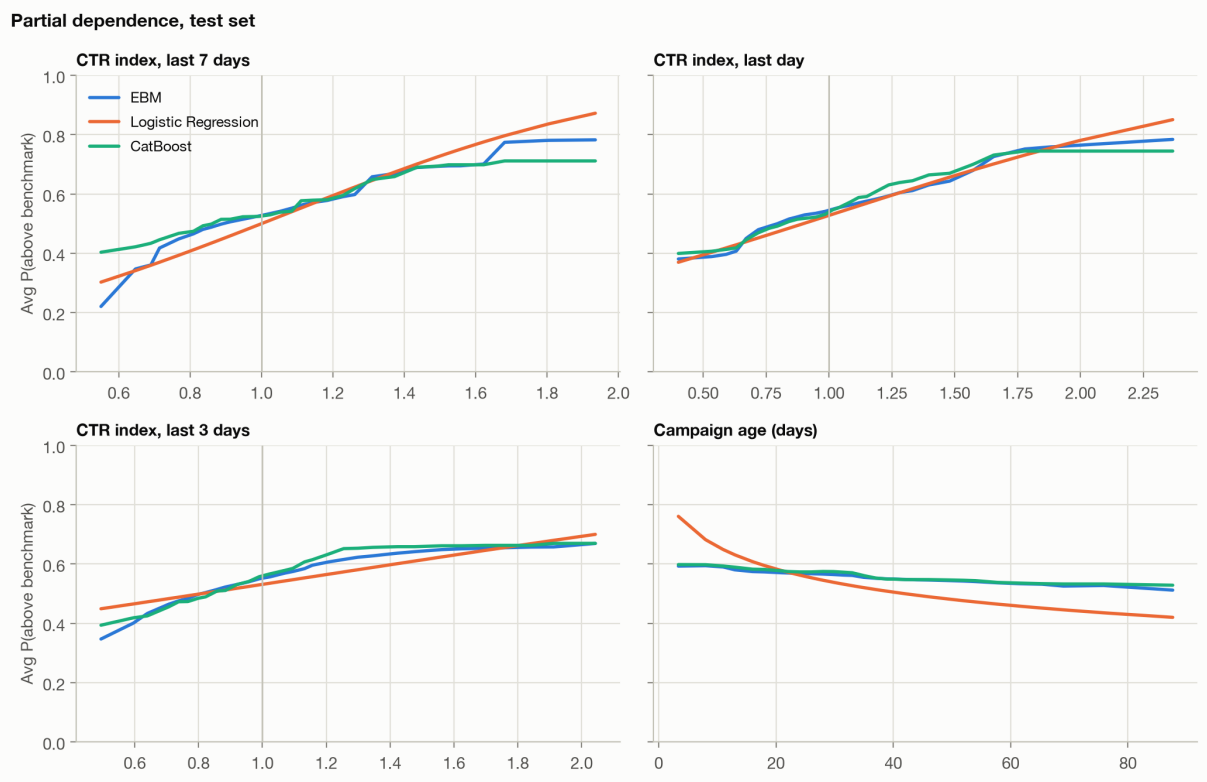


Figure 9. Partial dependence on the test set for the four most important numeric features, for all three models.

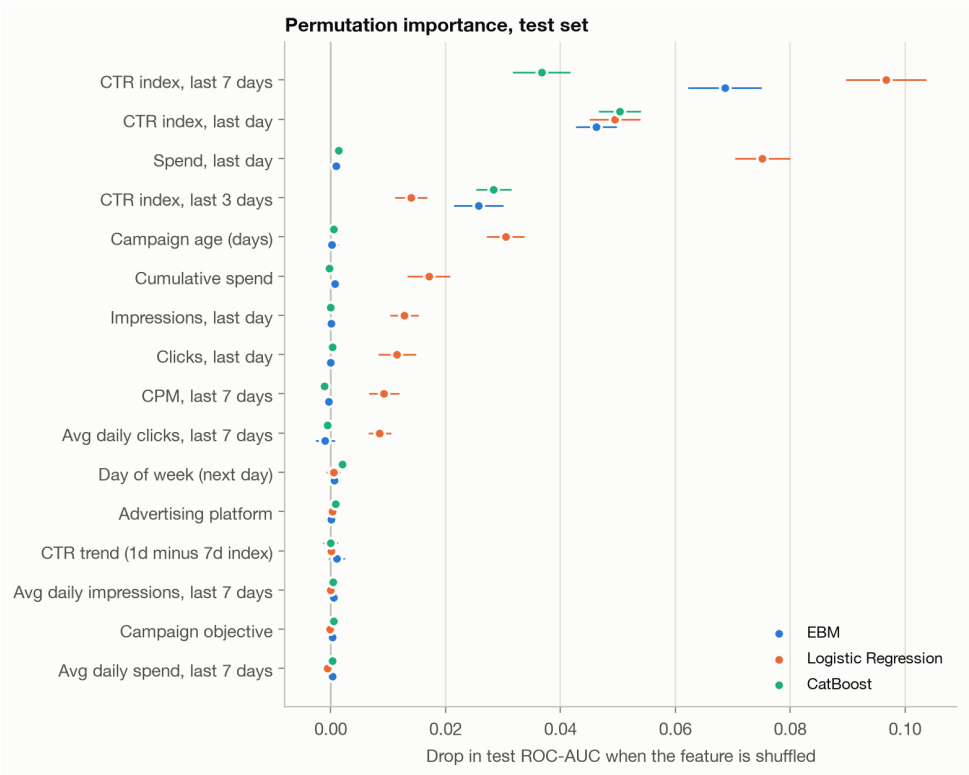


Figure 10. Permutation importance on the test set (mean and standard deviation over 10 shuffles).

6.7 Does the model agree with domain knowledge?

Broadly, yes, but this agreement carries limited weight. The simulation was built from the same assumptions, so these patterns were put there by design. Agreement shows that the model recovers them, not that they hold in real accounts. CTR persistence reflects the fact that an advert's relevance and audience fit change slowly. The decline with campaign age matches the long-documented wear-out of advertising under repeated exposure (Pechmann and Stewart, 1988). A weekday peak is plausible for search-led campaigns aimed at people at work, though it needs validation across additional campaigns and time periods. The EBM identifies persistence, campaign-age patterns and early-campaign variation; these associations do not establish their causes.

Two discrepancies are worth stating. First, the exploratory analysis showed opposite weekly patterns on the two platforms, but the EBM learned a single day-of-week curve, and none of its selected interactions involves platform and day of week. Its weekday effect is therefore an average that is slightly wrong for both platforms. EBM lets interactions be specified explicitly, so a platform × day-of-week term can be forced into the model. Second, no feature expresses spend relative to the campaign's own norm, so the model cannot directly distinguish changes in budget intensity from differences in campaign size. A ratio of last-day spend to seven-day average spend would be a natural addition. Both points illustrate the value of a glass-box model: the gaps show up in the explanations, whereas in a black box they would stay hidden.

7. Front-end dashboard and deployment

7.1 Dashboard

The trained models are served through a Streamlit dashboard (`app/streamlit_app.py`) with five tabs.

- **Predict.** The user either picks a campaign and date from the dataset or starts a blank campaign, then enters or edits the platform, objective, campaign age, total spend and the last seven days of spend, impressions and clicks. The dashboard returns the probability that tomorrow's CTR beats the benchmark, an "on track" or "at risk" verdict, the EBM's contribution chart, a short plain-language summary of what raises and lowers the chance, and the probabilities from the two comparison models (Figure 11). Editing the table turns this into a what-if tool, for example to see how a further drop in clicks would change the outlook.
- **Campaign watchlist.** Scores every campaign active on a chosen date, or every campaign in an uploaded CSV file, ranks them from most to least at risk and names the main negative factor for each (Figure 12). The list can be downloaded.
- **What the model learned.** Interactive versions of the importance chart, shape functions, interactions and partial dependence plots.
- **Model performance.** Test metrics with confidence intervals, ROC and precision-recall curves, confusion matrices and the full hyperparameter search.
- **Data overview.** Dataset size, CTR trends by platform and the CTR distribution by objective.

Chance tomorrow's CTR beats the benchmark

29%

⚠ **At risk:** likely to fall below the benchmark tomorrow.

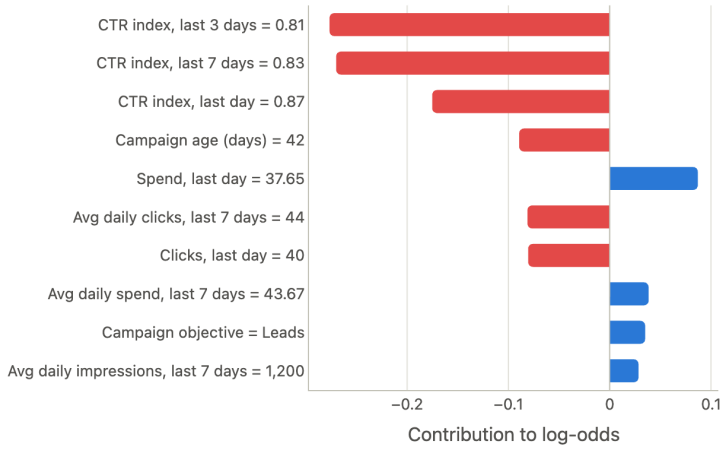
- Benchmark (Google Ads, Leads): **4.41%**
- CTR last day: **3.84%** · last 7 days: **3.67%**
- Predicting for: **Fri 26 Sep 2025**

What actually happened (original data): CTR 4.11%, **below** the benchmark.

All models

Model	P(above)	Threshold
EBM	29.3%	0.51
Logistic Regression	27.5%	0.57
CatBoost	30.0%	0.58

Why: the EBM's feature contributions



Raising the chance: Spend, last day = 37.65 (+0.09); Avg daily spend, last 7 days = 43.67 (+0.04); Campaign objective = Leads (+0.04)

Lowering the chance: CTR index, last 3 days = 0.81 (-0.28); CTR index, last 7 days = 0.83 (-0.27); CTR index, last day = 0.87 (-0.18)

Blue bars raise the chance of beating the benchmark; red bars lower it.

EBM is additive: intercept (-0.04) + the sum of all 26 contributions = log-odds -0.88, i.e. probability 29.3%. Every contribution is read directly off the model, not estimated after the fact.

Figure 11. Dashboard prediction for an at-risk Google Ads Leads campaign, with the EBM's explanation.

Campaigns scored

22

At risk tomorrow

7

On track

15

Status	Campaign	Platform	Objective	Last day	CTR, 7 days (%)	Benchmark (%)	P(above benchmark)	Biggest negative factor
⚠ At risk	cmp_031	Meta Ads	Awareness	2025-09-25	0.21	0.45	0.59%	CTR index, last 7 days
⚠ At risk	cmp_040	Meta Ads	Traffic	2025-09-25	0.83	1.17	2.22%	CTR index, last day
⚠ At risk	cmp_076	Meta Ads	Leads	2025-09-25	0.62	0.78	10.64%	CTR index, last day
⚠ At risk	cmp_066	Google Ads	Leads	2025-09-25	3.67	4.41	29.26%	CTR index, last 3 days
⚠ At risk	cmp_038	Google Ads	Leads	2025-09-25	4.02	4.41	40.12%	CTR index, last 7 days
⚠ At risk	cmp_015	Google Ads	Engagement	2025-09-25	1.72	1.7	45.96%	CTR index, last 3 days
⚠ At risk	cmp_063	Meta Ads	Engagement	2025-09-25	0.97	0.84	46.32%	CTR index, last day
✅ On track	cmp_069	Google Ads	Awareness	2025-09-25	0.49	0.54	52.25%	CTR index, last 7 days
✅ On track	cmp_004	Meta Ads	Sales	2025-09-25	0.75	0.74	60.2%	Avg daily clicks, last 7 days
✅ On track	cmp_037	Meta Ads	Awareness	2025-09-25	0.48	0.45	60.28%	Avg daily impressions, last 7

Figure 12. Campaign watchlist ranked by the chance of beating the benchmark tomorrow.

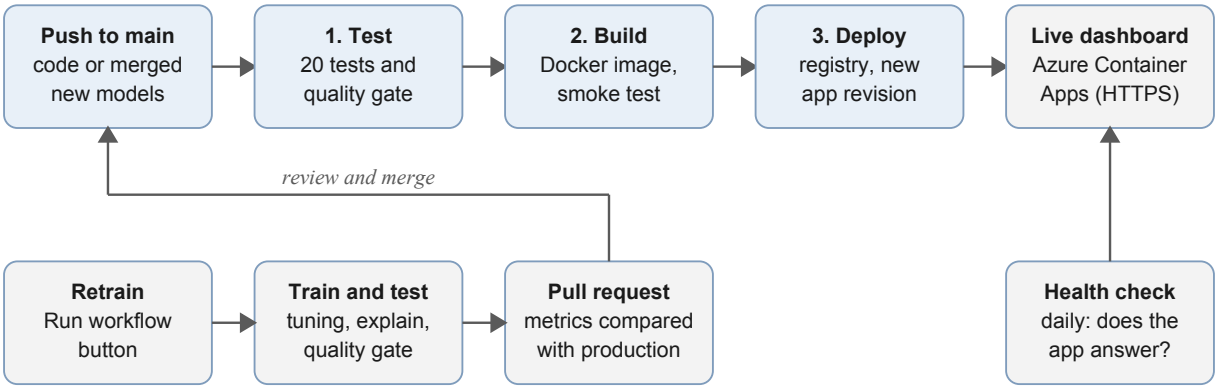
The dashboard calls the same feature-engineering code as the training pipeline. A test confirmed that, for 60 random campaign-days, the inputs it builds from a seven-day history are identical to those used in training.

7.2 Deployment and MLOps

The dashboard runs on Microsoft Azure as a Docker container on Azure Container Apps, which scales to zero when nobody is using it, so apart from about US$5 a month for the container registry it stays within the free allowance of a student subscription. It is publicly available at:

https://ctr-dashboard.purplefield-4d57f34d.eastasia.azurecontainerapps.io

After a one-time setup script (`deploy/azure_setup.sh`), everything is automated with three GitHub Actions workflows that follow common MLOps practice (Kreuzberger, Kühl and Hirschl, 2023), shown in Figure 13 and Table 9.



All steps run in GitHub Actions; only the initial Azure setup is run once by hand

Figure 13. Deployment and MLOps workflow. Blue steps run on every push to the main branch; grey steps run on demand, daily or in production.

Table 9. The three GitHub Actions workflows and their first runs.

Workflow	Trigger	Steps	First run
CI/CD ci-cd.yml	Every push to the main branch	Tests and quality gate; build and smoke-test the image; push it to the registry, roll out a new revision and check the live URL	Failed at the Azure sign-in (Table 11); after the fix, succeeded in about four minutes
Retrain retrain.yml	"Run workflow" button	Tune and train; explainability analysis; tests and quality gate on the new models; push them to a branch with a pull-request link and a metrics comparison	Succeeded in about three minutes
Health check health-check.yml	Daily at 03:00 UTC (08:30 in Sri Lanka)	Request the live app's health endpoint, allowing time for a cold start	Succeeded

Every push to the main branch runs `ci-cd.yml`. The tests treat the model as part of the software: besides checking the code, they confirm that the saved models still reproduce the test predictions in this report, that each EBM explanation adds up exactly to its prediction, and that features computed for a day do not change when all later days are removed. A quality gate then stops the release if the EBM falls below a test ROC-AUC of 0.80 or no longer beats the persistence baseline. Only an image that has passed these tests and rendered the dashboard inside its container is deployed (Figure 14). The retraining workflow (`retrain.yml`) runs the full training pipeline on the data in the repository and proposes the new models in a pull request, so a person reviews the metric changes before anything reaches production; merging the pull request triggers the normal CI/CD run. The first retraining run, on GitHub's Linux servers rather than the machine the models were first trained on, produced slightly

different models from the same data and seeds: the EBM's test ROC-AUC moved from 0.893 to 0.894 and CatBoost's from 0.895 to 0.892. Small platform-dependent changes like these are why new models are compared and reviewed before they are deployed rather than replacing the old ones automatically. Table 10 summarises how the deployment covers common MLOps practices.

Table 10. MLOps practices in the deployment.

Practice	Implementation
Versioning	Code, data, models and metrics are kept in git. Each Docker image and each Container Apps revision is tagged with its commit, which the dashboard shows as “App build”.
Automated testing	20 tests on data cleaning, the chronological split, look-ahead leakage, reproduction of the reported results, EBM additivity and the dashboard.
Quality gate	A release is blocked unless the EBM reaches a test ROC-AUC of at least 0.80 and beats the persistence baseline.
Continuous delivery	Test, then build and smoke-test the container, push it to Azure Container Registry, roll out a new revision and check the live URL.
Continuous training	A retraining workflow repeats tuning, evaluation and the quality gate, then proposes the models in a pull request that compares their metrics with production.
Monitoring	A daily health check of the live app, plus container logs and metrics in Azure Log Analytics.
Rollback	Re-running an earlier deploy job puts that commit's image back into service.
Security	GitHub signs in to Azure through OpenID Connect without a stored password, as an identity limited to one resource group; the app pulls images with a separate read-only identity.

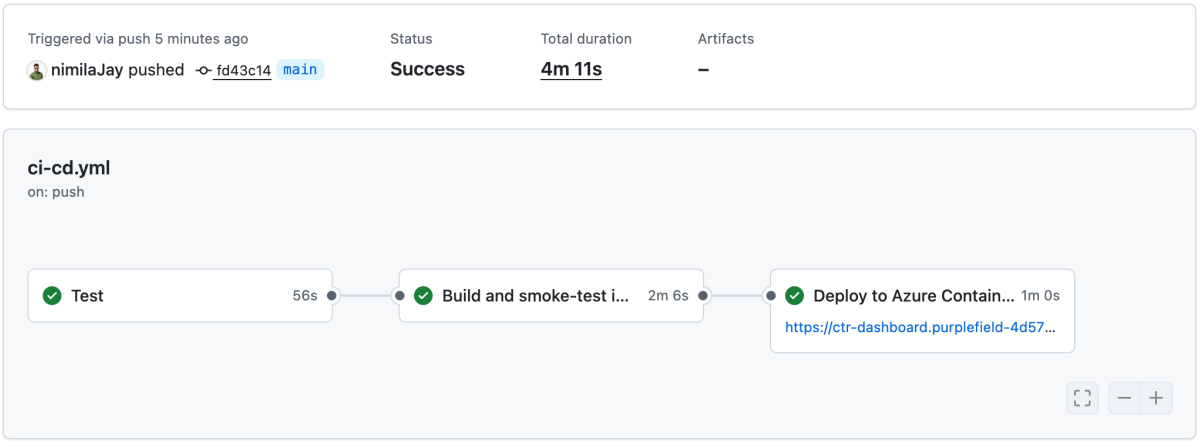


Figure 14. A CI/CD run in GitHub Actions: tests, then build and smoke test of the container image, then deployment to the production environment on Azure.

The set-up has clear limits, which are typical of ML systems in their first deployment (Sculley et al., 2015). Retraining is started by hand rather than by a schedule or by detected data drift, and the health check only confirms that the app answers, not that its predictions remain accurate. The training data is stored in the repository. Confidential account exports should instead be read from protected cloud storage by the retraining workflow.

7.3 Deployment challenges

Getting the deployment to work took several attempts. Table 11 lists the problems in the order they came up. Most of them come from the limits of a student subscription or from recent changes to Azure and GitHub, so anyone repeating the set-up is likely to meet them; all of the fixes are now part of the setup script.

Table 11. Problems met during deployment and how they were fixed.

Problem	Cause	Fix
Azure created an “express” Container Apps environment, which rejected the per-commit revision names	A new default in the Azure command-line tool	Ask for the standard (workload profiles) environment explicitly
The environment stayed in “Waiting” for 40 minutes in Malaysia West, and a second one could not be created	Azure for Students allows five regions and one Container Apps environment per subscription	Delete it and deploy to East Asia, where the environment was ready in about two minutes
Pushing the image failed after the registry was recreated in East Asia	DNS caches still pointed the registry's name at the deleted registry in Malaysia West	Registry names now include the region, so a new region gets a fresh name
GitHub Actions could not sign in to Azure (error AADSTS700213)	GitHub's sign-in tokens identify the repository by owner and repository IDs, not only by name	Trust the exact subject prefix that GitHub reports through its API
Workflows could not open pull requests	GitHub's default repository setting, kept for security	The retraining workflow pushes a branch and links a ready-made pull request for a person to open
Retrained models differed slightly from the originals	Floating-point differences between the Apple-silicon laptop and GitHub's Linux servers	Expected; the pull request compares new and production metrics before anything is deployed

None of these problems showed up in local tests; each appeared only when the pipeline ran against the real cloud services. Deployment code therefore needs to be exercised end to end, just like the model code, before it can be relied on.

8. Critical discussion

8.1 Limitations of the model

The model looks one day ahead and answers a yes/no question. It says nothing about how far above or below the benchmark a campaign will land, and a campaign hovering around its benchmark will flip between classes for reasons that are mostly noise. Its inputs are limited to a campaign's own recent delivery: creative content, audience targeting, placements, competitor activity, promotions and seasonality are all missing, and in practice they drive much of the variation in CTR. The misclassified case in Section 6.4 is typical of what the model cannot see. The gain over the persistence rule is real but modest, especially for hard yes/no labels, so the model is best seen as a way to rank and explain campaigns rather than as a precise forecast. Finally, a single threshold serves some segments worse than others (Section 5.3), and the model has been evaluated on one test period only.

8.2 Data quality

The most important limitation is that the results come from simulated data. The simulation encodes plausible mechanisms, but its noise is cleaner than real data and every effect the model found was put there by design. Evaluation also covers one dataset and one held-out time period, so performance on other campaigns and periods requires further validation. Data quality can be affected by changes in tracking and attribution, different click definitions (Meta's link clicks against all clicks), reporting time zones and currencies, platforms restating recent days, and small campaigns whose daily CTR is dominated by noise. The cleaning code handles duplicates, gaps, total rows and impossible values, and the 100-impression rule limits noisy labels. Consistent measurement definitions and ongoing monitoring remain necessary.

8.3 Risks of bias and unfairness

The dataset contains no personal data, so the usual concerns about protected characteristics do not apply directly. Bias can still arise at the level of campaigns. Small-budget campaigns have fewer impressions and noisier CTR, so their predictions are less reliable and they may be flagged more often for random reasons. Long-running campaigns contribute more campaign-days and therefore carry more weight in the median benchmark. There is also a feedback risk: if flagged campaigns are routinely paused, they stop producing data, and the model never learns whether they would have recovered. Performance also differs by objective (Table 8), so a team relying on the tool for Sales campaigns would miss more problems than one running Traffic campaigns. Reporting results by segment, as done here, is a first safeguard.

8.4 Real-world impact and ethical considerations

Used well, the tool helps a small team decide where to look first each morning and gives a reason that can be checked against what the team knows about the campaign. Used badly, it could replace judgement: a confident "on track" label might lead someone to ignore a campaign whose client brief has just changed. The predictions describe associations in historical data, not causes, so they should not be used to justify actions such as cutting a budget on the grounds that the model "shows" spend lowers CTR. The interpretable model helps with accountability here, because any decision based on it can be traced to specific contributions. On the data side, real use requires the account owner's permission, confidentiality for client names and figures, and compliance with the advertising platforms' terms of service. Because conditions on the platforms change constantly, the model would need regular retraining and its performance would have to be monitored over time.

9. Conclusion and future work

This project built an end-to-end pipeline that predicts whether a digital marketing campaign's CTR will beat its benchmark tomorrow and explains each prediction. On a simulated dataset of 4,818 campaign-days, an Explainable Boosting Machine matched the accuracy of CatBoost and logistic regression, outperformed a persistence rule, and produced explanations of the patterns in the dataset while exposing two gaps in the feature set. These results come from simulated campaigns, so real use requires validation on real account data.

The next steps follow directly from the discussion. The first priority is to repeat the evaluation on real account data. The feature set should gain a platform × day-of-week interaction and a relative-spend feature. Per-segment thresholds, and a regression or multi-day target, would make the predictions more useful in practice. Finally, the deployed dashboard already has automated tests, a quality gate, on-demand retraining and a daily health check; data-drift monitoring and a retraining schedule are the remaining steps towards production use.

References

Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.

Caruana, R., Lou, Y., Gehrke, J., Koch, P., Sturm, M. and Elhadad, N. (2015) 'Intelligible models for healthcare: predicting pneumonia risk and hospital 30-day readmission', *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1721–1730.

Efron, B. and Tibshirani, R.J. (1993) *An Introduction to the Bootstrap*. New York: Chapman & Hall.

Friedman, J.H. (2001) 'Greedy function approximation: a gradient boosting machine', *Annals of Statistics*, 29(5), pp. 1189–1232.

Hastie, T. and Tibshirani, R. (1990) *Generalized Additive Models*. London: Chapman & Hall.

Kreuzberger, D., Kühl, N. and Hirschl, S. (2023) 'Machine learning operations (MLOps): overview, definition, and architecture', *IEEE Access*, 11, pp. 31866–31879.

Lou, Y., Caruana, R. and Gehrke, J. (2012) 'Intelligible models for classification and regression', *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 150–158.

Lou, Y., Caruana, R., Gehrke, J. and Hooker, G. (2013) 'Accurate intelligible models with pairwise interactions', *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 623–631.

McMahan, H.B. et al. (2013) 'Ad click prediction: a view from the trenches', *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1222–1230.

Nori, H., Jenkins, S., Koch, P. and Caruana, R. (2019) 'InterpretML: a unified framework for machine learning interpretability', *arXiv preprint* arXiv:1909.09223.

Pechmann, C. and Stewart, D.W. (1988) 'Advertising repetition: a critical review of wearin and wearout', *Current Issues and Research in Advertising*, 11(1–2), pp. 285–329.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V. and Gulin, A. (2018) 'CatBoost: unbiased boosting with categorical features', *Advances in Neural Information Processing Systems*, 31.

Richardson, M., Dominowska, E. and Ragno, R. (2007) 'Predicting clicks: estimating the click-through rate for new ads', *Proceedings of the 16th International Conference on World Wide Web*, pp. 521–530.

Rudin, C. (2019) 'Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead', *Nature Machine Intelligence*, 1(5), pp. 206–215.

Sculley, D. et al. (2015) 'Hidden technical debt in machine learning systems', *Advances in Neural Information Processing Systems*, 28, pp. 2503–2511.

Youden, W.J. (1950) 'Index for rating diagnostic tests', *Cancer*, 3(1), pp. 32–35.

Appendix A. Reproducing the results

All results can be regenerated from the source code. Software used: Python 3.12.12, interpret 0.7.8, scikit-learn 1.9.1, CatBoost 1.2.10, pandas 3.1.0, Streamlit 1.66.1. From the project folder:

```
pip install -r requirements.txt

python -m src.train                # cleaning, features, tuning, evaluation

python -m src.explain              # explainability figures and tables

streamlit run app/streamlit_app.py # dashboard

pytest                             # tests and quality gate

bash deploy/azure_setup.sh        # one-time Azure setup; later deployments run in
GitHub Actions
```

Random seeds are fixed, and two consecutive runs produced identical data and metrics.

Table A1. Main source files.

File	Purpose
src/data.py	Loading, validation and cleaning; converter for platform exports
src/generate_data.py	Campaign dataset generation utility
src/features.py	Features, benchmark, target and chronological split
src/models.py	Model definitions and hyperparameter grids
src/train.py	Tuning, threshold selection, refitting and test evaluation
src/evaluate.py	Metrics, cluster bootstrap, segment analysis and evaluation plots
src/explain.py	EBM explanations, partial dependence and permutation importance
notebooks/01_eda.ipynb	Exploratory data analysis
app/streamlit_app.py	Interactive dashboard
tests/	Automated tests, including the deployment quality gate
Dockerfile	Container image of the dashboard
.github/workflows/	GitHub Actions: CI/CD to Azure, retraining and daily health check
deploy/azure_setup.sh	One-time Azure setup
report/build_report.py	Builds this report from the saved results